YAPAY ZEKÂ VE ADALET EĞİTİMİ · HAFTA 3

Kodlama ve Python'a Giriş: Temel Programlama

Kodlama nedir, Python nasıl çalışır ve programlamanın yapı taşları: değişkenler, veri tipleri, kurallar, koşullar, döngüler ve fonksiyonlar — hepsi örneklerle.

Teknik Giriş · ön bilgi gerektirmez

Ders: 90 dk

NEREDEN BURAYA?

Neden kod öğreniyoruz?

İlk iki hafta yapay zekânın **ne** olduğunu ve **nasıl** öğrendiğini konuştuk. Bir YZ sistemini gerçekten inceleyip sorgulayabilmek için ise onun dilini biraz anlamamız gerekiyor: **kod.**

Bugünün hedefi

Sizi yazılımcı yapmak değil — ama bir kod parçasını **okuyabilecek, çalıştırabilecek, mantığını anlayabilecek** temel seviyeye getirmek. Bugün programlamanın yapı taşlarını tek tek kuruyoruz.

BUGÜNÜN PLANI

90 dakikalık akış

Süre	Bölüm
0–12 dk	A · Kodlama nedir? Python nasıl çalışır? Ortam
12–35 dk	B · Değişkenler, veri tipleri, operatörler
35–45 dk	C · Yazım kuralları + D · Koşullar (if)
45–60 dk	E · Döngüler (for, while) + F · Listeler
60–78 dk	G · Fonksiyonlar
78–90 dk	H · Kütüphaneler + veri + YZ demosu + özet

Bölüm A

Kodlama Nedir? Python Nasıl Çalışır?

Önce temel fikri oturtalım.

TANIM

Kodlama nedir?

Kod / program

Bilgisayara verilen, **adım adım ve kesin talimatlar dizisi.** Bilgisayar bu talimatları sırayla, harfiyen uygular.

Kritik nokta: bilgisayar "akıllı" değildir; **tam olarak ne yazarsanız onu yapar** — eksiğini tamamlamaz, niyetinizi tahmin etmez. Bu yüzden kod **kesin** olmak zorundadır. Programlama, bir problemi bilgisayarın anlayacağı net adımlara bölmektir.

ÇALIŞMA MANTIĞI

Python nasıl çalışır?

Python bir **yorumlayıcı (interpreter)** dildir: kodunuzu **yukarıdan aşağı, satır satır** okur ve o an çalıştırır. Bir satırda hata varsa, oraya kadar çalışır, orada durur.

```
print("1. satır çalışır")
print("2. satır çalışır")
print("3. satır çalışır")
```

```
1. satır çalışır 2. satır çalışır 3. satır çalışır
```

Sıra önemlidir: bir değişkeni kullanmadan önce tanımlamış olmanız gerekir.

ORTAM

Nerede yazacağız? (Kurulumsuz)

Google Colab ile kodu doğrudan tarayıcıda yazıp çalıştırabiliriz — kurulum yok, ücretsiz. Kodu küçük **hücreler** hâlinde yazarsınız; her hücreyi ▶ (veya `Shift+Enter`) ile çalıştırır, sonucu hemen altında görürsünüz.

Hata almak normaldir

Kırmızı yazı bir şeyi bozduğunuz anlamına gelmez; bilgisayarın "bunu anlamadım" demesidir. Genelde son satır sorunu söyler; okur, düzeltir, yeniden çalıştırırız.

Bölüm B

Değişkenler, Veri Tipleri, Operatörler

Programın malzemesi: veriyi tutmak ve işlemek.

DEĞİŞKENLER

Değeri saklamak: değişken

Değişken, bir değere verdiğimiz isimdir. `=` işareti "atama" yapar: sağdaki değeri soldaki isme koyar.

```
isim = "Ayşe"      # metin
yas  = 22           # sayı
yas  = yas + 1      # değeri güncelle → 23
print(isim, yas)
```

```
Ayşe 23
```

İsimlendirme kuralları

Harf ve alt çizgiyle başlar (`ad` , `toplam_puan`); boşluk ve Türkçe karakter kullanılmaz; büyük/küçük harf farklıdır (`yas` ≠ `Yas`).

VERİ TİPLERİ

Verinin türleri

Tip	Adı	Örnek
Tam sayı	int	`5`, `-3`, `200`
Ondalık	float	`3.14`, `0.5`
Metin	str	`"Ayşe"`, `"merhaba"`
Mantıksal	bool	`True` (doğru), `False` (yanlış)

```
print(type(5), type(3.14), type("Ayşe"), type(True))
```

```
<class 'int'> <class 'float'> <class 'str'> <class 'bool'>
```

OPERATÖRLER · 1

Matematiksel işlemler

İşlem	Sembol	Örnek → Sonuç
Toplama / çıkarma	`+` `-`	`7 + 2` → 9
Çarpma / bölme	`*` `/`	`7 / 2` → 3.5
Tam bölüm	`//`	`7 // 2` → 3
Kalan (mod)	`%`	`7 % 2` → 1
Üs	`**`	`2 ** 3` → 8

OPERATÖRLER · 2

Karşılaştırma ve mantık

Bu operatörler **doğru/yanlış (bool)** üretir — kararların temeli.

Karşılaştırma	Anlam	Mantıksal	Anlam
`==`	eşit mi?	`and`	ikisi de doğru
`!=`	eşit değil mi?	`or`	en az biri doğru
`>` `<`	büyük / küçük	`not`	tersi
`>=` `<=`	büyük-eşit / küçük-eşit		

```
print(5 > 3,   10 == 10,   (5 > 3) and (2 < 1))
```

```
True True False
```

Bölüm C

Yazım Kuralları

Python'un dilbilgisi: bunlara uymazsak kod çalışmaz.

PYTHON'UN KURALLARI

Dört temel yazım kuralı

- **Girinti (indentation):** Bir bloğun içindeki satırlar içeriden başlar (genelde 4 boşluk). Python blokları girintiyle anlar — süsleme değil, **zorunluluktur.**
- **İki nokta (:):** `if`, `for`, `def` gibi satırlar iki noktayla biter; sonraki blok girintilenir.
- **Büyük/küçük harf duyarlı:** `Print` ≠ `print`. Kelimeler tam yazılmalı.
- **Yorum (#):** `#` ile başlayan satır çalışmaz; sadece insan için nottur.

DİKKAT

En sık iki hata

IndentationError

Girinti yanlış/eksik. Blok içindeki satırlar hizalı ve içeriden başlamalı.

SyntaxError

Yazım hatası: eksik iki nokta, kapanmamış parantez ya da tırnak. Genelde hata mesajı satırı gösterir.

Bu hatalar normaldir; okumayı öğrenince hızla düzeltirsiniz.

Bölüm D

Koşullar (Karar Kuralları)

Bilgisayara "eğer... ise... değilse..." dedirtmek.

İF / ELİF / ELSE

Koşula göre karar

Bir koşul doğruysa bir şey, değilse başka bir şey yaparız. Bu, bir modelin "eşikle karar vermesine" çok benzer.

```
puan = 72
if puan >= 85:
    print("Pekiyi")
elif puan >= 50:
    print("Geçti")
else:
    print("Kaldı")
```

```
Geçti
```

Okunuşu

`if` (eğer) → `elif` (değilse eğer) → `else` (hiçbiri değilse). Python koşulları yukarıdan aşağı dener, ilk doğru olanı çalıştırır.

Bölüm E

Döngüler

Aynı işi tekrar tekrar yaptırmak.

FOR DÖNGÜSÜ

Bir listenin her elemanı için

`for`, bir listedeki her eleman için aynı işi tekrarlar — tek tek yazmaya gerek kalmaz.

```
adaylar = ["Ayşe", "Zeynep", "Elif"]
for aday in adaylar:
    print("Merhaba", aday)
```

```
Merhaba Ayşe Merhaba Zeynep Merhaba Elif
```

RANGE VE WHİLE

Sayarak ve koşullu döngü

`range` ile belli sayıda tekrar:

```
for i in range(3):
    print("Tekrar", i)
```

```
Tekrar 0 Tekrar 1 Tekrar 2
```

`while`: koşul doğru olduğu sürece:

```
sayac = 3
while sayac > 0:
    print(sayac)
    sayac = sayac - 1
```

```
3 2 1
```

Bölüm F

Veri Yapıları

Birden çok değeri düzenli tutmak.

LİSTE

Sıralı bir koleksiyon

```
notlar = [70, 85, 60]
print(notlar[0])        # ilk eleman → 70
print(len(notlar))      # eleman sayısı → 3
notlar.append(90)       # sona ekle
print(notlar)           # [70, 85, 60, 90]
```

Önemli

Python **0'dan** saymaya başlar: ilk eleman `[0]`, ikinci `[1]` … Bu, en sık karıştırılan kuraldır.

SÖZLÜK (DİCT)

Anahtar–değer çiftleri

Bazı verileri "isimle" saklamak isteriz. **Sözlük**, her değere bir **anahtar** verir:

```
aday = {"isim": "Ayşe", "deneyim": 5, "sehir": "İzmir"}
print(aday["isim"])       # Ayşe
print(aday["deneyim"])   # 5
```

```
Ayşe 5
```

Bir veri satırını (bir adayı) böyle temsil edebiliriz — YZ verisinin küçük bir hâli.

Bölüm G

Fonksiyonlar

Kodu paketlemek ve yeniden kullanmak.

FONKSİYON NEDİR?

Bir işi bir isme paketlemek

Fonksiyon, bir işi yapan kod bloğuna verdiğimiz isimdir. Bir kez `def` ile tanımlarsınız, sonra adıyla defalarca **çağırırsınız.**

```
def selamla():
    print("Merhaba!")

selamla()      # fonksiyonu çağır
selamla()      # tekrar çağır
```

```
Merhaba! Merhaba!
```

PARAMETRE VE RETURN

Girdi alan, sonuç döndüren fonksiyon

Parametre: fonksiyona verdiğimiz girdi. **return**: fonksiyonun geri verdiği sonuç.

```
def topla(a, b):
    sonuc = a + b
    return sonuc

x = topla(7, 5)      # a=7, b=5
print(x)              # 12
```

```
12
```

Girdi → İşlem → Çıktı

Fonksiyon tam da modeller gibi çalışır: bir **girdi** alır, bir **çıktı** döndürür. Bu benzetmeyi aklınızda tutun.

NEDEN ÖNEMLİ?

Fonksiyonlar neden hayat kurtarır?

Tekrar kullanım

Bir işi bir kez yazarsınız, yüz kez çağırırsınız. Kopyala-yapıştır yok.

Düzen

Karmaşık bir programı, her biri bir iş yapan küçük parçalara bölersiniz.

Bizim için asıl önemi

YZ araçlarını kullanmak, aslında **başkalarının yazdığı fonksiyonları çağırmaktır.** `pipeline(...)`, `read_csv(...)` — hepsi birer fonksiyon. Yazmayı değil, çağırmayı ve çıktısını okumayı öğreniyoruz.

Bölüm H

Kütüphaneler ve İlk Uygulama

Hazır araçları çağırmak.

KÜTÜPHANELER

Hazır araç kutuları: import

Kütüphane (library), birilerinin yazıp paylaştığı fonksiyon topluluğudur. `import` ile çağırıp kullanırız — sıfırdan yazmayız.

```
import pandas as pd            # veri tabloları için
from transformers import pipeline  # hazır YZ modelleri için
```

Fikir

Python'un gücü buradan gelir: neredeyse her iş için hazır bir kütüphane vardır. Biz bu araçları **bilinçli kullanan** kişiler olacağız.

Bir tabloyu okumak

```
import pandas as pd
veri = pd.read_csv("adaylar.csv")   # dosyayı oku (bir fonksiyon!)
print(veri.head())                  # ilk satırlar
print(veri["ise_alindi"].value_counts())  # kaç Evet/Hayır
```

```
deneyim egitim sehir ise_alindi 0 5 Lisans İzmir Evet 1 8 Y.Lisans Ankara Hayır Evet 120 Hayır 80
```

İLK YZ DEMOSU

Dört satırla yapay zekâ

```
from transformers import pipeline
model = pipeline("sentiment-analysis")   # hazır modeli çağır
print(model("Bu eğitim gerçekten harika!"))
print(model("Hiçbir şey anlamadım."))
```

```
[{'label': 'POSITIVE', 'score': 0.99}] [{'label': 'NEGATIVE', 'score': 0.97}]
```

Örnek çıktı. Gördüğünüz her şey bugün öğrendiklerimiz: import, fonksiyon çağırma, girdi → çıktı.

TOPARLAYALIM

Bugün ne öğrendik?

- **Kodlama** = bilgisayara kesin, adım adım talimat vermek; Python satır satır çalışır.
- **Değişken** değer saklar; **veri tipleri** (int, float, str, bool) ve **operatörler** onu işler.
- **Yazım kuralları** (girinti, iki nokta, harf duyarlılığı) uyulması zorunlu dilbilgisidir.
- **Koşullar** (if/elif/else) karar verir; **döngüler** (for/while) tekrarlar; **liste** ve **sözlük** veriyi tutar.
- **Fonksiyonlar** işi paketler; **kütüphaneler** hazır fonksiyonlar sunar — YZ araçları da böyle.

BAŞVURU

Terimler sözlüğü

Terim	Anlamı	Terim	Anlamı
Değişken	Değere verilen isim	Fonksiyon	İsimlenmiş kod bloğu
Veri tipi	int/float/str/bool	Parametre	Fonksiyona girdi
Operatör	+, ==, and…	return	Fonksiyonun çıktısı
Koşul (if)	Karar kuralı	Kütüphane	Hazır fonksiyon topluluğu
Döngü	Tekrar (for/while)	import	Kütüphaneyi çağırma
Liste / sözlük	Veri yapıları	Girinti	Blokları belirleyen boşluk

BU HAFTANIN ALIŞTIRMASI

Evde deneyin

Küçük görevler

- Bir değişkene yaşınızı atayın, 10 ekleyip yazdırın.
- `if` ile: puan 50'den büyükse "Geçti" yazdırın.
- Bir liste yapın, `for` ile her elemanını yazdırın.
- `topla(a, b)` fonksiyonunu yazın ve iki sayıyı toplatın.
- Duygu analizi demosuna 3 kendi cümlenizi verin.

HAFTA 4 · ÖNİZLEME

Yapay Zekâ Nasıl Çalışır (1): Veri

Kodun temelini kurduk. Gelecek hafta bu araçları veriye yöneltiyoruz: bir modele hangi veriyi veriyoruz, "özellik" ve "etiket" nedir, ve önyargı bu tabloya nereden giriyor? Gerçek bir istihdam verisiyle inceleyeceğiz.